

Project: Reti neurali e stochastic gradient descent

ottimizzazione stocastica e apprendimento supervisionato

1. Obiettivi della dispensa

Questa dispensa introduce le reti neurali artificiali come caso di studio per un corso di metodi computazionali, con particolare attenzione al ruolo dello stochastic gradient descent.

Gli obiettivi sono cinque:

1. comprendere la struttura elementare del neurone artificiale;
2. descrivere come i neuroni vengano composti in reti a più strati;
3. formalizzare il problema dell'apprendimento come problema di minimizzazione di una funzione obiettivo;
4. distinguere tra gradient descent completo e stochastic gradient descent;
5. chiarire perché questo caso di studio riguarda soprattutto l'ottimizzazione stocastica, più che la dinamica stocastica del fenomeno modellato.

Questa precisazione è importante. In molti modelli del corso il rumore entra nella dinamica del sistema studiato. Nel caso delle reti neurali, invece, la stocasticità entra soprattutto nella procedura di ottimizzazione: il problema è la minimizzazione di una loss, e il rumore nasce dal sottocampionamento casuale dei dati durante l'aggiornamento dei parametri.

2. Motivazione generale

Le reti neurali artificiali sono modelli parametrici flessibili che permettono di approssimare relazioni complesse tra input e output. Sono oggi usate in compiti molto diversi:

- classificazione;
- regressione;
- riconoscimento di pattern;
- elaborazione del linguaggio;
- visione artificiale;
- previsione di serie temporali.

Dal punto di vista matematico, una rete neurale non è altro che una funzione

$$f(x; \theta),$$

dove:

- x rappresenta l'input;
- θ rappresenta l'insieme dei parametri del modello, cioè pesi e bias;
- f produce un output che vogliamo rendere il più possibile vicino al dato osservato.

L'apprendimento consiste quindi nel trovare i parametri θ che minimizzano una funzione di errore.

3. Il neurone artificiale

3.1 Struttura elementare

Un neurone artificiale riceve un vettore di input

$$x = (x_1, \dots, x_d),$$

e associa a ciascun input un peso

$$w = (w_1, \dots, w_d).$$

Calcola poi una combinazione lineare degli input:

$$z = \sum_{j=1}^d w_j x_j + b,$$

dove b e' un bias.

L'output del neurone si ottiene applicando una funzione di attivazione ϕ :

$$y = \phi(z).$$

Questa e' la struttura fondamentale di quasi tutte le reti neurali.

3.2 Interpretazione dei pesi

I pesi controllano l'importanza relativa delle diverse componenti dell'input. In modo molto elementare:

- un peso positivo tende a rafforzare il contributo di una variabile;
- un peso negativo tende a contrastarlo;
- il bias trasla la soglia di attivazione del neurone.

Dal punto di vista geometrico, il termine lineare

$$z = w \cdot x + b$$

definisce un iperpiano nello spazio degli input.

3.3 Funzioni di attivazione

La funzione di attivazione introduce non linearità. Senza questa, una rete composta da molti strati resterebbe globalmente equivalente a una sola trasformazione lineare.

Esempi classici sono:

Funzione soglia

$$\phi(z) = \begin{cases} 1 & \text{se } z \geq 0, \\ 0 & \text{se } z < 0. \end{cases}$$

Sigmoide

$$\phi(z) = \frac{1}{1 + e^{-z}}.$$

Tangente iperbolica

$$\phi(z) = \tanh(z).$$

ReLU

$$\phi(z) = \max(0, z).$$

Per una dispensa introduttiva, e' sufficiente capire che la funzione di attivazione consente di costruire relazioni non lineari tra input e output.

4. Il percettrone

4.1 Modello elementare di classificazione

Il percettrone e' uno dei modelli storici più semplici. Esso realizza una classificazione binaria sulla base della regola

$$y = \phi(w \cdot x + b),$$

dove ϕ e' una funzione soglia.

In questo caso il modello separa lo spazio degli input in due regioni tramite un iperpiano.

4.2 Limite del percettrone

Il percettrone semplice e' un classificatore lineare. Funziona bene solo quando le classi sono linearmente separabili.

Questa limitazione motiva il passaggio a reti con uno o più strati nascosti, in cui le trasformazioni successive permettono di costruire frontiere decisionali molto più complesse.

5. Architettura di una rete neurale

5.1 Strati della rete

Una rete neurale feed-forward è organizzata in strati:

- strato di input;
- uno o più hidden layers;
- strato di output.

Ogni strato riceve l'output dello strato precedente e applica una trasformazione del tipo

$$h^{(\ell+1)} = \phi \left(W^{(\ell)} h^{(\ell)} + b^{(\ell)} \right),$$

dove:

- $h^{(\ell)}$ e' il vettore delle attivazioni dello strato ℓ ;
- $W^{(\ell)}$ e' la matrice dei pesi;
- $b^{(\ell)}$ e' il vettore dei bias;
- ϕ e' la funzione di attivazione.

5.2 Reti feed-forward

In una rete feed-forward l'informazione scorre in una sola direzione:

$$\text{input} \rightarrow \text{hidden layers} \rightarrow \text{output}.$$

Non vi sono cicli. Questo e' il modello più naturale per introdurre l'apprendimento supervisionato.

5.3 Reti ricorrenti

Le note menzionano anche reti ricorrenti. In esse l'output di uno stato può influenzare l'evoluzione successiva del sistema. In forma astratta:

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b).$$

Le reti ricorrenti sono importanti per dati sequenziali e serie temporali. Tuttavia, per questa dispensa il focus principale resta sulle reti feed-forward e sul problema dell'ottimizzazione dei parametri.

6. Il problema dell'apprendimento supervisionato

6.1 Dataset

Supponiamo di avere un dataset di esempi

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

dove:

- x_i è l'input del campione i ;
- y_i è l'output corretto associato a quell'input.

Il problema consiste nel trovare i parametri θ della rete in modo che

$$f(x_i; \theta)$$

sia il più vicino possibile a y_i per tutti gli esempi.

6.2 Loss su un singolo campione

Per misurare l'errore su un dato campione, si introduce una loss

$$\ell_i(\theta) = \ell(f(x_i; \theta), y_i).$$

Per esempio, in regressione, una scelta classica è l'errore quadratico:

$$\ell_i(\theta) = \frac{1}{2}(f(x_i; \theta) - y_i)^2.$$

In classificazione si usano spesso altre funzioni di perdita, ma il punto concettuale resta identico: quantificare quanto l'output della rete si discosti dal target.

6.3 Loss complessiva

La loss totale sul dataset si può scrivere come media:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_i(\theta).$$

L'apprendimento consiste quindi nel risolvere il problema di ottimizzazione

$$\min_{\theta} L(\theta).$$

Questa e' la formulazione centrale dell'intera dispensa.

7. Gradient descent

7.1 Idea di base

Per minimizzare $L(\theta)$ si usa spesso il gradiente. Se θ e' un vettore di parametri, il gradiente

$$\nabla L(\theta)$$

indica la direzione di massima crescita della funzione obiettivo.

Quindi, per diminuire la loss, si deve muovere θ nella direzione opposta:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t),$$

dove $\eta > 0$ e' il learning rate.

7.2 Interpretazione geometrica

L'algoritmo esegue una discesa nel paesaggio della funzione obiettivo:

- se il gradiente è grande, ci si trova in una zona ripida;
- se il gradiente è piccolo, si è vicini a un punto stazionario;
- il learning rate controlla la lunghezza del passo.

Questa e' un'idea molto generale di ottimizzazione, non specifica delle reti neurali.

7.3 Problema computazionale

Il vero limite del gradient descent completo è che, per calcolare

$$\nabla L(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\theta),$$

bisogna sommare il contributo di tutti i dati del dataset a ogni iterazione.

Se N è molto grande, questo diventa costoso.

8. Dal gradient descent allo stochastic gradient descent

8.1 Idea del sottocampionamento casuale

Per ridurre il costo computazionale, invece di usare tutto il dataset a ogni passo, si può scegliere casualmente un solo campione o un piccolo sottoinsieme di campioni.

Nel caso di un singolo campione scelto a caso, l'aggiornamento diventa

$$\theta_{t+1} = \theta_t - \eta \nabla \ell_{i_t}(\theta_t),$$

dove i_t è un indice estratto casualmente.

Questa è la forma più semplice dello stochastic gradient descent.

8.2 Mini-batch SGD

Più spesso si usa un mini-batch B_t di dimensione m :

$$\theta_{t+1} = \theta_t - \eta \frac{1}{m} \sum_{i \in B_t} \nabla \ell_i(\theta_t).$$

In questo caso:

- l'aggiornamento è meno costoso del gradiente completo;
- il gradiente stimato è meno rumoroso che nel caso di un solo campione;
- l'algoritmo è particolarmente adatto a implementazioni efficienti.

8.3 Perché SGD è stocastico

La stocasticità non proviene dalla rete in sé, ma dalla scelta casuale del campione o del mini-batch.

Quindi, a ogni passo, il gradiente usato nell'aggiornamento non coincide esattamente con il gradiente completo, ma con una sua stima casuale.

Questo produce una dinamica del tipo

$$\theta_{t+1} = \theta_t - \eta (\nabla L(\theta_t) + \xi_t),$$

dove ξ_t rappresenta il rumore introdotto dal sottocampionamento.

Questa è la formulazione più importante dell'intero modulo dal punto di vista del corso.

9. Interpretazione dello SGD come ottimizzazione stocastica

9.1 Vantaggi

Lo stochastic gradient descent ha almeno tre vantaggi fondamentali:

1. ogni aggiornamento costa meno;
2. permette di lavorare su dataset grandi;
3. il rumore può aiutare a evitare un'eccessiva sensibilità a dettagli locali del paesaggio di ottimizzazione.

9.2 Trade-off

Tuttavia introduce anche un compromesso:

- il passo è più economico;
- ma la direzione seguita è più rumorosa.

Quindi la traiettoria dei parametri non è liscia come nel gradient descent completo. In compenso, l'algoritmo può essere più efficiente e più robusto in pratica.

9.3 Punto metodologico

Questo è il vero messaggio da sottolineare agli studenti:

- il gradient descent classico cerca di seguire esattamente il gradiente della loss totale;
- lo SGD sostituisce il gradiente vero con una stima casuale;
- l'algoritmo diventa quindi un processo stocastico nello spazio dei parametri.

Per questo motivo il modulo si inserisce molto bene in un corso di metodi computazionali, anche se non è un modello stocastico del fenomeno osservato, ma dell'algoritmo di apprendimento.

10. Ruolo del learning rate

Il learning rate η controlla l'ampiezza del passo.

Se η è troppo piccolo:

- l'apprendimento è molto lento.

Se η è troppo grande:

- la traiettoria può oscillare;
- la loss può anche aumentare;
- il metodo può diventare instabile.

In presenza di rumore da mini-batch, la scelta di η diventa ancora più delicata.

11. Obiettivo empirico e generalizzazione

Finora abbiamo parlato della minimizzazione della loss sul dataset di addestramento. Ma l'obiettivo reale non è soltanto adattarsi bene ai dati osservati. Si vuole anche ottenere buona capacità di generalizzazione, cioè buone prestazioni su dati non visti.

Questo introduce una distinzione importante tra:

- training error;
- test error.

Una rete può ridurre molto bene la loss di training e tuttavia generalizzare male. Questo problema è uno dei motivi per cui l'ottimizzazione non esaurisce da sola la teoria dell'apprendimento statistico.

12. Pseudocodice del gradient descent completo

Supponiamo di avere un dataset di N osservazioni e parametri iniziali θ_0 .

Input

- dataset $\{(x_i, y_i)\}_{i=1}^N$
- parametri iniziali θ
- learning rate η
- numero di iterazioni T

Pseudocodice

1. inizializza θ
2. per $t = 1, \dots, T$:
 - calcola il gradiente completo

$$\nabla L(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\theta)$$

- aggiorna i parametri

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

3. restituisci i parametri finali

Questo algoritmo è concettualmente semplice, ma può essere costoso per dataset grandi.

13. Pseudocodice dello stochastic gradient descent

Input

- dataset $\{(x_i, y_i)\}_{i=1}^N$
- parametri iniziali θ

- learning rate η
- numero di epoche E

Pseudocodice

1. inizializza θ
2. per ogni epoca:
 - mescola casualmente i dati
 - per ogni campione o mini-batch:
 - calcola il gradiente locale
 - aggiorna

$$\theta \leftarrow \theta - \eta \nabla \ell_B(\theta)$$

3. restituisci i parametri finali

Qui $\nabla \ell_B(\theta)$ indica il gradiente calcolato sul mini-batch.

14. Perché questo è un buon case study per il corso

Questo modulo ha una funzione molto precisa nel corso.

Primo, mostra una forma diversa di stocasticità: non quella della dinamica del sistema, ma quella della procedura numerica.

Secondo, collega in modo molto chiaro:

- modello parametrico;
- funzione obiettivo;
- gradiente;
- algoritmo di ottimizzazione;
- sottocampionamento casuale.

Terzo, fornisce un ponte naturale tra matematica applicata, informatica e machine learning.

15. Schema del laboratorio

15.1 Laboratorio 1 - Il neurone artificiale

Obiettivo

Implementare un neurone semplice e visualizzare l'effetto di pesi e bias.

Attività

1. definire input, pesi e bias;
2. calcolare

$$z = w \cdot x + b;$$

3. applicare una funzione di attivazione;
4. osservare come cambia l'output al variare dei parametri.

Domande guida

- che effetto ha il bias?
- cosa cambia con una funzione di attivazione lineare o non lineare?
- che significato geometrico ha il vettore dei pesi?

15.2 Laboratorio 2 - percettrone e classificazione lineare

Obiettivo

Usare un neurone come classificatore binario.

Attività

1. costruire un dataset bidimensionale semplice;
2. scegliere pesi iniziali;
3. osservare la frontiera decisionale;
4. aggiornare i pesi in modo elementare.

Domande guida

- quando i dati sono separabili linearmente?
- cosa succede quando non lo sono?
- perché un solo neurone non basta in generale?

15.3 Laboratorio 3 - Gradient descent su una loss semplice

Obiettivo

Minimizzare una loss quadratica in un modello elementare.

Attività

1. scegliere un dataset di regressione;
2. definire una loss quadratica;
3. calcolare il gradiente;
4. iterare il gradient descent;
5. visualizzare la discesa della loss.

Domande guida

- il learning rate influisce sulla convergenza?
- il metodo converge sempre?
- cosa succede se il passo è troppo grande?

15.4 Laboratorio 4 - Stochastic gradient descent

Obiettivo

Confrontare gradient descent completo e SGD.

Attività

1. implementare i due algoritmi;
2. usare lo stesso dataset;
3. confrontare la traiettoria della loss;
4. studiare l'effetto della dimensione del mini-batch.

Domande guida

- perché lo SGD produce una traiettoria più rumorosa?
- quando il rumore aiuta e quando ostacola?
- quanto costa ogni iterazione nei due casi?

16. Estensioni naturali

Questa dispensa introduce solo il nucleo del problema. Le estensioni naturali sono molte.

1. **Backpropagation:** Per reti con più strati, il calcolo efficiente dei gradienti richiede la backpropagation.
2. **Mini-batch e shuffle:** Si può studiare in dettaglio come il mescolamento casuale dei dati influenzi l'apprendimento.
3. **Ottimizzatori avanzati:** Si possono introdurre algoritmi come momentum, RMSProp, Adam.
4. **Regularization:** Si può aggiungere penalizzazione sui parametri per controllare overfitting e complessità del modello.
5. **Validazione:** Si può distinguere tra training set, validation set e test set.

17. Conclusione

Le reti neurali forniscono un contesto molto chiaro per introdurre l'ottimizzazione stocastica. Il problema fondamentale è la minimizzazione di una loss su un dataset, e lo stochastic gradient descent realizza questa minimizzazione sostituendo il gradiente completo con una stima casuale calcolata su un sottoinsieme dei dati.

Dal punto di vista del corso, questo modulo è importante perché mostra una forma di stocasticità diversa da quella incontrata in altri casi di studio. Qui il rumore non è principalmente nel fenomeno modellato, ma nell'algoritmo che apprende i parametri del modello.

Per questo motivo, le reti neurali e lo SGD costituiscono un eccellente esempio di ottimizzazione numerica stocastica.

18. Bibliografia minima

1. Rosenblatt, F. (1958). The perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6), 386-408.
 2. Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning Representations by Back-Propagating Errors. *Nature*, 323, 533-536.
 3. Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
 4. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
 5. Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
-

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python i concetti principali della dispensa:

1. neurone artificiale;
2. perceptron;
3. funzione di loss;
4. gradient descent;
5. stochastic gradient descent.

L'obiettivo non è costruire un framework avanzato di machine learning, ma fornire una guida leggibile che possa servire sia a chi usa Python sia a chi usa altri linguaggi. Per questo motivo il codice è volutamente semplice:

- poche librerie;
- funzioni corte;
- passaggi espliciti;
- nessuna dipendenza speciale oltre a librerie molto comuni;
- struttura leggibile quasi come pseudocodice.

Il punto didattico importante è che qui non vogliamo “nascondere” l'algoritmo dentro librerie già pronte. Vogliamo vedere esplicitamente:

- come si calcola l'output di un neurone;
- come si misura l'errore;
- come si aggiornano i pesi;
- dove entra la stocasticità nello SGD.

A.1 Librerie minime

Per questa appendice bastano:

```
import random
import math
import matplotlib.pyplot as plt
```

Quindi:

- `random` serve per inizializzare parametri e per lo shuffle dei dati;
- `math` serve per funzioni elementari come esponenziale;
- `matplotlib.pyplot` serve per grafici semplici.

Non è necessario usare `numpy` in una prima implementazione.

A.2 Rappresentare i dati

Supponiamo di avere un dataset supervisionato formato da coppie input-output.

Per esempio, in un problema semplice di regressione o classificazione binaria:

```
python id="9ve2mo" dataset = [ ([0.0, 0.0], 0.0), ([0.0, 1.0], 1.0), ([1.0, 0.0], 1.0), ([1.0, 1.0], 1.0) ]
```

Qui:

- il primo elemento di ogni coppia è il vettore di input;
- il secondo elemento è il target.

In generale, un campione si rappresenta come

```
python id="e4jtyn" (x, y)
```

dove `x` è una lista di numeri e `y` è il valore da prevedere.

A.3 Il neurone artificiale

A.3.1 Combinazione lineare

Dato un input

$$x = (x_1, \dots, x_d)$$

e un vettore di pesi

$$w = (w_1, \dots, w_d),$$

il neurone calcola

$$z = \sum_{j=1}^d w_j x_j + b.$$

In Python:

```
“python id=“jddx1a” def linear_combination(x, w, b): z = 0.0
for j in range(len(x)):
    z += w[j] * x[j]

z += b
return z
```

Questa funzione è il cuore del neurone.

A.3.2 Funzioni di attivazione

Sigmoide

```
```python id="gy9hqo"
def sigmoid(z):
 return 1.0 / (1.0 + math.exp(-z))
```

ReLU

```
python id="mue83m" def relu(z): if z > 0.0: return z
else: return 0.0
```

Soglia

```
python id="w1jq2g" def threshold(z): if z >= 0.0: return
1.0 else: return 0.0
```

### A.3.3 Output di un neurone

Un neurone completo si ottiene combinando la parte lineare e l'attivazione:

```
“python id=“rvszik” def neuron_output(x, w, b, activation=“sigmoid”): z =
linear_combination(x, w, b)

if activation == "sigmoid":
 return sigmoid(z)
elif activation == "relu":
 return relu(z)
elif activation == "threshold":
 return threshold(z)
else:
 return z
```

Questa funzione è già sufficiente per sperimentare con un neurone singolo.

## ## A.4 Inizializzazione dei parametri

Per iniziare, i pesi e il bias possono essere estratti casualmente da un intervallo piccolo.

```
```python id="vpfz4v"
def initialize_parameters(input_dimension):
    w = []
    for j in range(input_dimension):
        w.append(random.uniform(-0.5, 0.5))

    b = random.uniform(-0.5, 0.5)

    return w, b
```

Questa scelta è semplice e sufficiente per un primo laboratorio.

A.5 Funzione di loss

A.5.1 Errore quadratico su un singolo dato

Per un singolo campione, possiamo usare la loss quadratica:

$$\ell = \frac{1}{2}(\hat{y} - y)^2,$$

dove $\hat{y}$ è la previsione del modello.

In Python:

```
python id="505xdi" def squared_loss(y_hat, y):    return 0.5 *
(y_hat - y) ** 2
```

A.5.2 Loss media sul dataset

```
“python id="ht08y4" def average_loss(dataset, w, b, activation="sigmoid"):
losses = []
```

```
for x, y in dataset:
    y_hat = neuron_output(x, w, b, activation=activation)
    losses.append(squared_loss(y_hat, y))
```

```
return sum(losses) / len(losses)
```

Questa funzione calcola la loss media su tutto il dataset.

A.6 Perceptron e classificazione

Se si usa l'attivazione a soglia, il neurone diventa un classificatore lineare elementare.

```
```python id="gjiuxm"
def perceptron_predict(x, w, b):
 return neuron_output(x, w, b, activation="threshold")
```

Questa funzione è utile per far vedere come un singolo neurone realizzi una separazione lineare.

## A.7 Gradient descent in un modello semplice

Per mantenere il codice molto chiaro, conviene partire da un neurone con attivazione lineare o sigmoide e loss quadratica.

### A.7.1 Caso più semplice: neurone lineare

Consideriamo il caso

$$\hat{y} = w \cdot x + b.$$

In questo caso la loss è

$$\ell = \frac{1}{2}(\hat{y} - y)^2.$$

Il gradiente rispetto ai pesi è

$$\frac{\partial \ell}{\partial w_j} = (\hat{y} - y)x_j,$$

e rispetto al bias

$$\frac{\partial \ell}{\partial b} = \hat{y} - y.$$

### A.7.2 Predizione lineare

```
python id="u2u268" def linear_model_output(x, w, b): return
linear_combination(x, w, b)
```

### A.7.3 Gradiente su un singolo campione

```
“python id="v97r24" def single_sample_gradient(x, y, w, b): y_hat = li-
near_model_output(x, w, b) error = y_hat - y
```

```

grad_w = []
for j in range(len(w)):
 grad_w.append(error * x[j])

grad_b = error

return grad_w, grad_b

```

Questa funzione è molto importante, perché rende esplicito come si calcola il gradiente.

## ## A.8 Gradient descent completo

Nel gradient descent completo, si fa la media dei gradienti su tutto il dataset.

```

```python id="jlwm7u"
def full_gradient(dataset, w, b):
    grad_w = [0.0 for _ in range(len(w))]
    grad_b = 0.0

    for x, y in dataset:
        sample_grad_w, sample_grad_b = single_sample_gradient(x, y, w, b)

        for j in range(len(w)):
            grad_w[j] += sample_grad_w[j]

        grad_b += sample_grad_b

    n = len(dataset)

    for j in range(len(w)):
        grad_w[j] /= n

    grad_b /= n

    return grad_w, grad_b

```

A.8.1 Un passo di gradient descent

```

“python id="af48up" def gradient_descent_step(dataset, w, b, learning_rate):
    grad_w, grad_b = full_gradient(dataset, w, b)

    new_w = []
    for j in range(len(w)):
        new_w.append(w[j] - learning_rate * grad_w[j])

    new_b = b - learning_rate * grad_b

```

```
return new_w, new_b
```

A.8.2 Allenamento completo con gradient descent

```
```python id="wvf9v0"
def train_gradient_descent(dataset, input_dimension, learning_rate, epochs):
 w, b = initialize_parameters(input_dimension)

 history_loss = []

 for epoch in range(epochs):
 loss = average_loss_linear(dataset, w, b)
 history_loss.append(loss)

 w, b = gradient_descent_step(dataset, w, b, learning_rate)

 results = {
 "w": w,
 "b": b,
 "history_loss": history_loss
 }

 return results
```

Per usare questa funzione, serve anche una versione della loss media per il modello lineare:

```
“python id="2w7vpe" def average_loss_linear(dataset, w, b): losses = []
for x, y in dataset:
 y_hat = linear_model_output(x, w, b)
 losses.append(squared_loss(y_hat, y))

return sum(losses) / len(losses)
```

## ## A.9 Stochastic gradient descent

Nel caso dello SGD, a ogni passo si usa solo un campione o un piccolo mini-batch.

### ## A.9.1 Un passo di SGD con un solo campione

```
```python id="w1t1jd"
def stochastic_gradient_step(x, y, w, b, learning_rate):
    grad_w, grad_b = single_sample_gradient(x, y, w, b)
```

```

new_w = []
for j in range(len(w)):
    new_w.append(w[j] - learning_rate * grad_w[j])

new_b = b - learning_rate * grad_b

return new_w, new_b

```

A.9.2 Allenamento con SGD

```

“python id=“wlvr6” def train_sgd(dataset, input_dimension, learning_rate,
epochs): w, b = initialize_parameters(input_dimension)

history_loss = []

for epoch in range(epochs):
    random.shuffle(dataset)

    for x, y in dataset:
        w, b = stochastic_gradient_step(x, y, w, b, learning_rate)

    loss = average_loss_linear(dataset, w, b)
    history_loss.append(loss)

results = {
    "w": w,
    "b": b,
    "history_loss": history_loss
}

return results

```

Questa funzione mostra molto bene dove entra la stocasticità:

- * i dati vengono mescolati casualmente;
- * l'aggiornamento usa un campione alla volta;
- * la traiettoria dei parametri è quindi rumorosa.

A.10 Mini-batch SGD

Un passo ulteriore consiste nell'usare piccoli gruppi di campioni.

A.10.1 Creare mini-batch

```

```python id="1kj7pj"
def create_mini_batches(dataset, batch_size):

```

```

batches = []

for start in range(0, len(dataset), batch_size):
 end = start + batch_size
 batch = dataset[start:end]
 batches.append(batch)

return batches

```

### A.10.2 Gradiente di un mini-batch

```

“python id=“ygh9nw” def mini_batch_gradient(batch, w, b): grad_w = [0.0
for _ in range(len(w))] grad_b = 0.0

for x, y in batch:
 sample_grad_w, sample_grad_b = single_sample_gradient(x, y, w, b)

 for j in range(len(w)):
 grad_w[j] += sample_grad_w[j]

 grad_b += sample_grad_b

m = len(batch)

for j in range(len(w)):
 grad_w[j] /= m

grad_b /= m

return grad_w, grad_b

```

### ## A.10.3 Un passo di mini-batch SGD

```

```python id="btfchr"
def mini_batch_step(batch, w, b, learning_rate):
    grad_w, grad_b = mini_batch_gradient(batch, w, b)

    new_w = []
    for j in range(len(w)):
        new_w.append(w[j] - learning_rate * grad_w[j])

    new_b = b - learning_rate * grad_b

    return new_w, new_b

```

A.10.4 Allenamento con mini-batch

```
“python id=“vb6pws” def train_mini_batch_sgd(dataset, input_dimension,
learning_rate, epochs, batch_size): w, b = initialize_parameters(input_dimension)

history_loss = []

for epoch in range(epochs):
    random.shuffle(dataset)
    batches = create_mini_batches(dataset, batch_size)

    for batch in batches:
        w, b = mini_batch_step(batch, w, b, learning_rate)

    loss = average_loss_linear(dataset, w, b)
    history_loss.append(loss)

results = {
    "w": w,
    "b": b,
    "history_loss": history_loss
}

return results
```

A.11 Confronto tra gradient descent e SGD

Una delle esercitazioni più utili è confrontare le traiettorie della loss.

```
```python id="n16o34"
def plot_loss_histories(loss_gd, loss_sgd, loss_mini_batch=None):
 epochs = list(range(len(loss_gd)))

 plt.plot(epochs, loss_gd, label="gradient descent")
 plt.plot(epochs, loss_sgd, label="sgd")

 if loss_mini_batch is not None:
 plt.plot(epochs, loss_mini_batch, label="mini-batch sgd")

 plt.xlabel("epoca")
 plt.ylabel("loss")
 plt.title("Confronto tra algoritmi di ottimizzazione")
 plt.legend()
 plt.show()
```

Esempio:

```

“python id=“p69cxp” dataset = [([0.0, 0.0], 0.0), ([0.0, 1.0], 1.0), ([1.0, 0.0],
1.0), ([1.0, 1.0], 2.0)]

results_gd = train_gradient_descent(dataset=dataset, input_dimension=2,
learning_rate=0.1, epochs=50)

results_sgd = train_sgd(dataset=dataset, input_dimension=2, learning_rate=0.1, epochs=50)

results_mb = train_mini_batch_sgd(dataset=dataset, input_dimension=2, learning_rate=0.1, epochs=50, batch_size=2)

plot_loss_histories(results_gd[“history_loss”], results_sgd[“history_loss”], results_mb[“history_loss”])

```

#### ## A.12 Cosa osservare nei grafici

Dal punto di vista didattico, i grafici servono a far vedere tre cose:

1. il gradient descent completo tende ad avere una traiettoria più regolare;
2. lo SGD mostra oscillazioni più visibili;
3. il mini-batch sta tipicamente in una posizione intermedia.

Questo è esattamente il modo più semplice per visualizzare la differenza tra gradiente completo e mini-batch.

#### ## A.13 Estensione a una rete con uno strato nascosto

Per una prima dispensa, non è necessario implementare una rete completa con backpropagation.

Una rete con uno strato nascosto fa questo:

1. prende l'input  $x$ ;
2. calcola le attivazioni dello hidden layer;
3. usa quelle attivazioni come input dello strato di output;
4. confronta l'output finale con il target;
5. aggiorna tutti i parametri tramite gradienti.

Dal punto di vista pratico, questa estensione richiede la backpropagation. Per il corso, però, non è necessario.

#### ## A.14 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo così:

1. import delle librerie;
  2. funzioni di base:
- ```

* `linear_combination`

```

```

    * attivazioni
    * `neuron_output`
3. funzioni per la loss:

    * `squared_loss`
    * `average_loss`
4. funzioni per il modello lineare:

    * `linear_model_output`
    * `single_sample_gradient`
    * `full_gradient`
5. ottimizzazione:

    * `gradient_descent_step`
    * `stochastic_gradient_step`
    * `mini_batch_step`
6. allenamento:

    * `train_gradient_descent`
    * `train_sgd`
    * `train_mini_batch_sgd`
7. grafici;
8. blocco finale con esempi.

```

Per esempio:

```

```python id="42u01y"
if __name__ == "__main__":
 dataset = [
 ([0.0, 0.0], 0.0),
 ([0.0, 1.0], 1.0),
 ([1.0, 0.0], 1.0),
 ([1.0, 1.0], 2.0)
]

 results_gd = train_gradient_descent(
 dataset=dataset,
 input_dimension=2,
 learning_rate=0.1,
 epochs=50
)

 results_sgd = train_sgd(
 dataset=dataset,
 input_dimension=2,
 learning_rate=0.1,

```



```

 epochs=50
)

 plot_loss_histories(
 results_gd["history_loss"],
 results_sgd["history_loss"]
)

 print("Pesi finali GD:", results_gd["w"], results_gd["b"])
 print("Pesi finali SGD:", results_sgd["w"], results_sgd["b"])

```

### A.15 Perché questa appendice è utile

Questa appendice ha due vantaggi principali.

Primo, mostra che le idee fondamentali di una rete neurale e dello SGD possono essere implementate con strumenti molto elementari. Non serve alcun framework avanzato per capire la logica del problema.

Secondo, la struttura del codice è abbastanza vicina a Python reale da essere eseguibile quasi subito, ma abbastanza esplicita da poter essere letta come pseudocodice da chi usa altri linguaggi.

### A.16 Conclusione dell'appendice

Il messaggio metodologico di questa appendice è semplice ma importante: una rete neurale, almeno al livello introduttivo, può essere letta come una funzione parametrica che viene addestrata minimizzando una loss. Lo stochastic gradient descent realizza questa minimizzazione usando stime casuali del gradiente ottenute da singoli campioni o mini-batch.

Per questo motivo, anche un'implementazione molto semplice è sufficiente per vedere il cuore del problema: la stocasticità non è tanto nella rete, quanto nell'algoritmo di ottimizzazione che ne aggiorna i parametri.